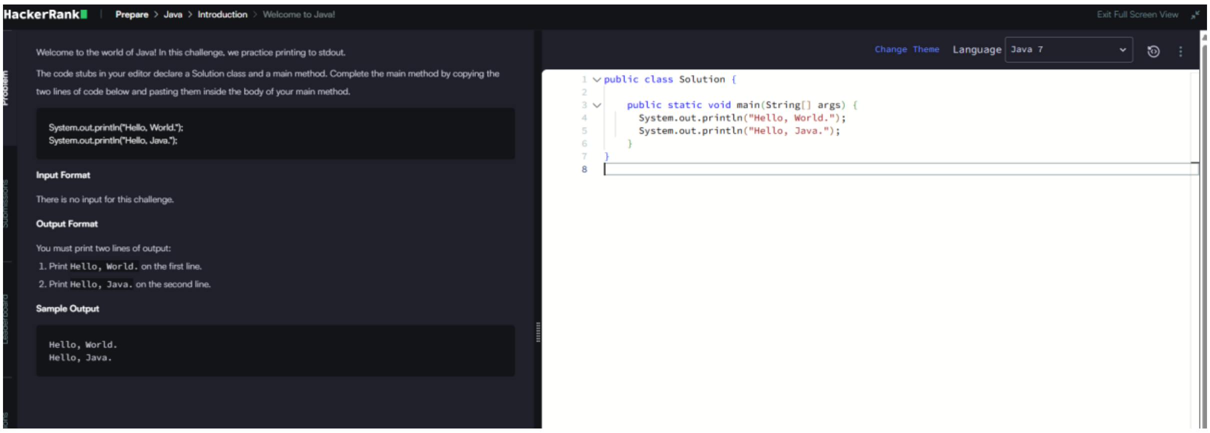
BANDLAPALLI BHANUTEJA REDDY—192325016

JAVA –HACKER RANK


HackerRank | Prepare > Java > Introduction > Welcome to Java!

Welcome to the world of Java! In this challenge, we practice printing to stdout.

The code stubs in your editor declare a Solution class and a main method. Complete the main method by copying the two lines of code below and pasting them inside the body of your main method.

```
System.out.println("Hello, World.");
System.out.println("Hello, Java.");
```

Input Format

There is no input for this challenge.

Output Format

You must print two lines of output:

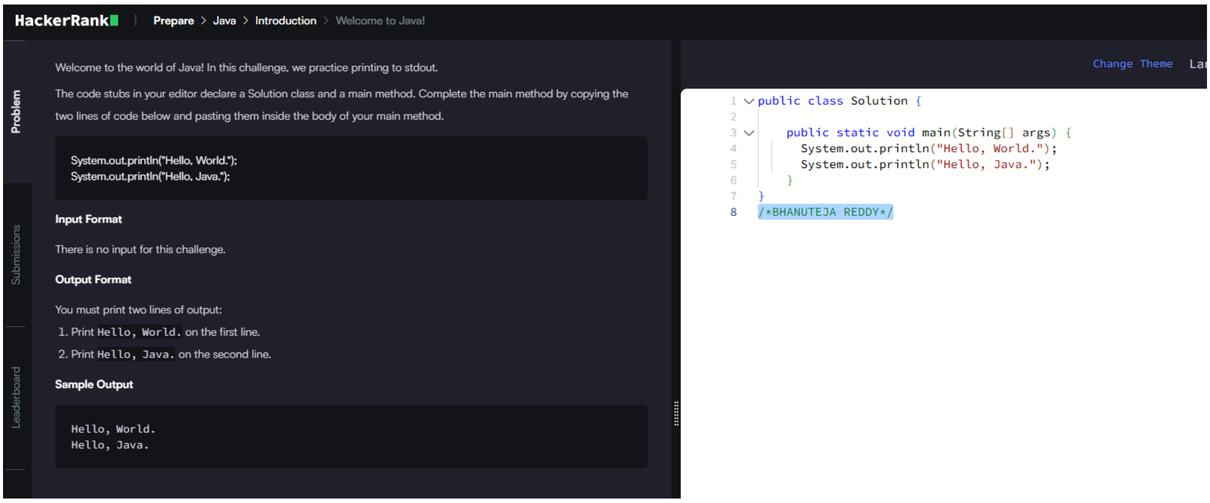
1. Print `Hello, World.` on the first line.
2. Print `Hello, Java.` on the second line.

Sample Output

```
Hello, World.
Hello, Java.
```

Change Theme Language Java 7

```java
public class Solution {

    public static void main(String[] args) {
        System.out.println("Hello, World.");
        System.out.println("Hello, Java.");
    }
}
```


HackerRank | Prepare > Java > Introduction > Welcome to Java!

Welcome to the world of Java! In this challenge, we practice printing to stdout.

The code stubs in your editor declare a Solution class and a main method. Complete the main method by copying the two lines of code below and pasting them inside the body of your main method.

```
System.out.println("Hello, World.");
System.out.println("Hello, Java.");
```

Input Format

There is no input for this challenge.

Output Format

You must print two lines of output:

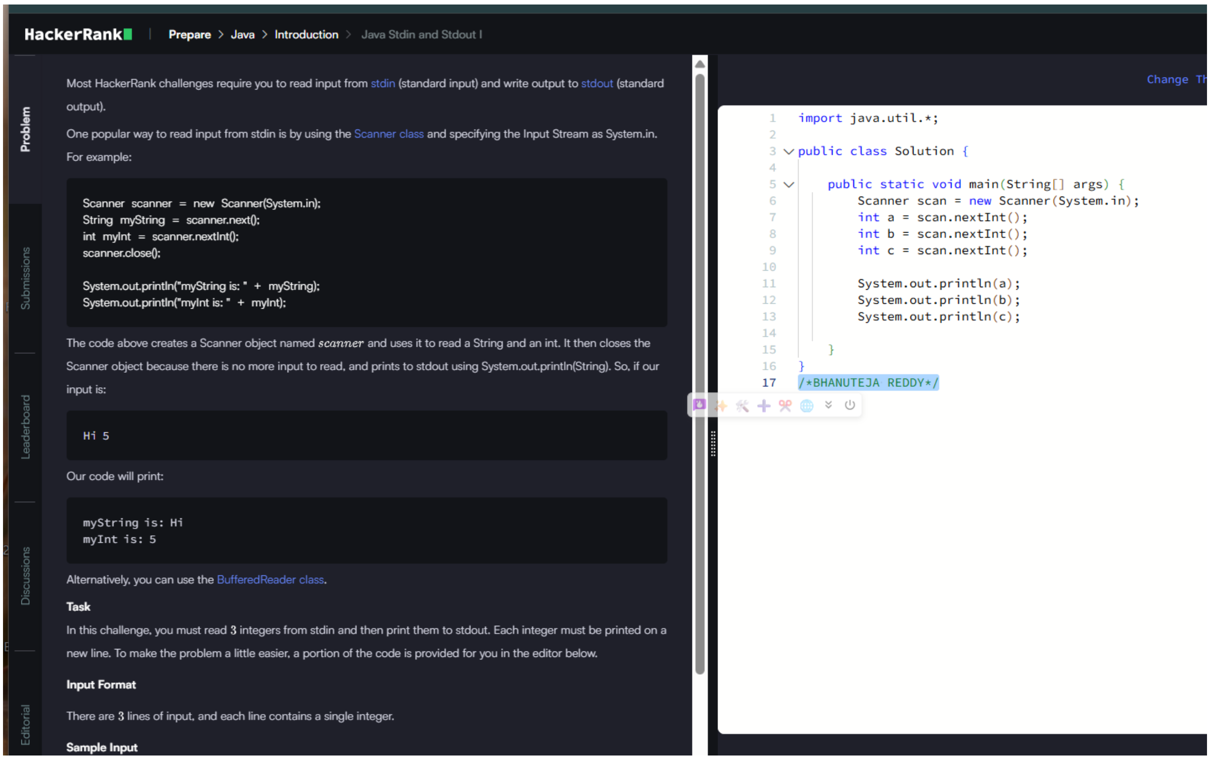
1. Print `Hello, World.` on the first line.
2. Print `Hello, Java.` on the second line.

Sample Output

```
Hello, World.
Hello, Java.
```

Change Theme

```java
public class Solution {

    public static void main(String[] args) {
        System.out.println("Hello, World.");
        System.out.println("Hello, Java.");
    }
}
/*BHANUTEJA REDDY*/
```


HackerRank | Prepare > Java > Introduction > Java Stdin and Stdout I

Most HackerRank challenges require you to read input from stdin (standard input) and write output to stdout (standard output).

One popular way to read input from stdin is by using the Scanner class and specifying the Input Stream as System.in. For example:

```
Scanner scanner = new Scanner(System.in);
String myString = scanner.next();
int myInt = scanner.nextInt();
scanner.close();

System.out.println("myString is: " + myString);
System.out.println("myInt is: " + myInt);
```

The code above creates a Scanner object named *scanner* and uses it to read a String and an int. It then closes the Scanner object because there is no more input to read, and prints to stdout using System.out.println(String). So, if our input is:

```
Hi 5
```

Our code will print:

```
myString is: Hi
myInt is: 5
```

Alternatively, you can use the BufferedReader class.

Task

In this challenge, you must read 3 integers from stdin and then print them to stdout. Each integer must be printed on a new line. To make the problem a little easier, a portion of the code is provided for you in the editor below.

Input Format

There are 3 lines of input, and each line contains a single integer.

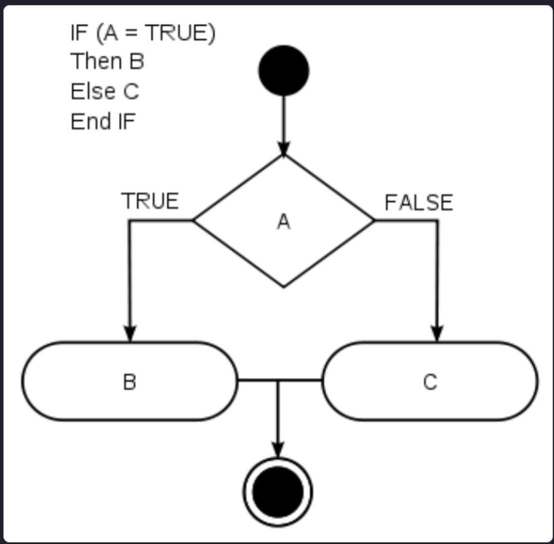
Sample Input

```java
import java.util.*;

public class Solution {

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int a = scan.nextInt();
        int b = scan.nextInt();
        int c = scan.nextInt();

        System.out.println(a);
        System.out.println(b);
        System.out.println(c);

    }
}
/*BHANUTEJA REDDY*/
```

HackerRank | Prepare > Java > Introduction > Java If-Else

Problem | Submissions | Leaderboard | Discussions | Editorial

In this challenge, we test your knowledge of using if-else conditional statements to automate decision-making processes. An if-else statement has the following logical flow:



Source: Wikipedia

Change Theme | Language

```java
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.regex.*;

public class Solution {
    private static final Scanner scanner = new Scanner(System.in);

    public static void main(String[] args) {
        int N = scanner.nextInt();
        scanner.skip("(\r\n|[\n\r\u2028\u2029\u0085])?");

        if(N%2!=0){
            System.out.println("Weird");
        }
        else if(N>=2 && N<=5)
        {
            System.out.println("Not Weird");
        }
        else if(N>=6 && N<=20)
        {
            System.out.println("Weird");
        }
        else
        {
            System.out.println("Not Weird");
        }

        scanner.close();
    }
}
/*BHANUTEJA REDDY*/
```

HackerRank | Prepare > Java > Introduction > Java Stdin and Stdout II

Problem | Submissions | Leaderboard | Discussions | Editorial

In this challenge, you must read an integer, a double, and a String from stdin, then print the values according to the instructions in the Output Format section below. To make the problem a little easier, a portion of the code is provided for you in the editor.

Note: We recommend completing Java Stdin and Stdout I before attempting this challenge.

Input Format

There are three lines of input:

1. The first line contains an integer.
2. The second line contains a double.
3. The third line contains a String.

Output Format

There are three lines of output:

1. On the first line, print `String:` followed by the unaltered String read from stdin.
2. On the second line, print `Double:` followed by the unaltered double read from stdin.
3. On the third line, print `Int:` followed by the unaltered integer read from stdin.

To make the problem easier, a portion of the code is already provided in the editor.

Note: If you use the nextLine() method immediately following the nextInt() method, recall that nextInt() reads integer tokens; because of this, the last newline character for that line of integer input is still queued in the input buffer and the next nextLine() will be reading the remainder of the integer line (which is empty).

Sample Input

```
42
3.1415
Welcome to HackerRank's Java tutorials!
```

Sample Output

```
String: Welcome to HackerRank's Java tutorials!
Double: 3.1415
Int: 42
```

Change Theme | Language

```java
import java.util.Scanner;

public class Solution {

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int i = scan.nextInt();
        double d=scan.nextDouble();
        scan.nextLine();

        String s=scan.nextLine();

        System.out.println("String: " + s);
        System.out.println("Double: " + d);
        System.out.println("Int: " + i);
    }
}
/*BHANUTEJA REDDY*/
```

HackerRank | Prepare > Java > Introduction > Java Output Formatting

Problem | Submissions | Leaderboard | Discussions | Editorial

Java's System.out.printf function can be used to print formatted output. The purpose of this exercise is to test your understanding of formatting output using printf.

To get you started, a portion of the solution is provided for you in the editor; you must format and print the input to complete the solution.

Input Format

Every line of input will contain a String followed by an integer.
Each String will have a maximum of 10 alphabetic characters, and each integer will be in the inclusive range from 0 to 999.

Output Format

In each line of output there should be two columns:
The first column contains the String and is left justified using exactly 15 characters.
The second column contains the integer, expressed in exactly 3 digits; if the original input has less than three digits, you must pad your output's leading digits with zeroes.

Sample Input

```
java 100
cpp 65
python 50
```

Sample Output

```
================================
java           100
cpp            065
python         050
================================
```

Explanation

Each String is left-justified with trailing whitespace through the first 15 characters. The leading digit of the integer is the 16^{th} character, and each integer that was less than 3 digits now has leading zeroes.

Change Theme Language Java 7

```java
import java.util.Scanner;

public class Solution {

    public static void main(String[] args) {
            Scanner sc=new Scanner(System.in);
            System.out.println("================================");
            for(int i=0;i<3;i++){
                String s1=sc.next();
                int x=sc.nextInt();
System.out.printf("%-15s%03d%n", s1,x);
            }
            System.out.println("================================");

    }
}
/*BHANUTEJA REDDY*/
```

HackerRank | Prepare > Java > Introduction > Java Loops I

Exit Full Scre

Problem | Submissions | Leaderboard | Discussions | Editorial

Objective

In this challenge, we're going to use loops to help us do some simple math.

Task

Given an integer, N, print its first 10 multiples. Each multiple $N \times i$ (where $1 \leq i \leq 10$) should be printed on a new line in the form: `N x i = result`.

Input Format

A single integer, N.

Constraints

- $2 \leq N \leq 20$

Output Format

Print 10 lines of output; each line i (where $1 \leq i \leq 10$) contains the $result$ of $N \times i$ in the form:
`N x i = result`.

Sample Input

```
2
```

Sample Output

```
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
2 x 10 = 20
```

Change Theme Language Java 7

```java
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.regex.*;


public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));

        int N = Integer.parseInt(bufferedReader.readLine().trim());
        for(int i=1;i<=10;i++){
            System.out.println(N+" x " + i+ " = "+(N*i));
        }

        bufferedReader.close();
    }
}
/*BHANUTEJA REDDY*/
```

HackerRank | Prepare > Java > Introduction > Java Loops II

Problem | Submissions | Leaderboard | Discussions | Editorial

We use the integers a, b, and n to create the following series:

$$(a + 2^0 \cdot b), (a + 2^0 \cdot b + 2^1 \cdot b), \ldots, (a + 2^0 \cdot b + 2^1 \cdot b + \ldots + 2^{n-1} \cdot b)$$

You are given q queries in the form of a, b, and n. For each query, print the series corresponding to the given a, b, and n values as a single line of n space-separated integers.

Input Format

The first line contains an integer, q, denoting the number of queries.
Each line i of the q subsequent lines contains three space-separated integers describing the respective a_i, b_i, and n_i values for that query.

Constraints

- $0 \le q \le 500$
- $0 \le a, b \le 50$
- $1 \le n \le 15$

Output Format

For each query, print the corresponding series on a new line. Each series must be printed in order as a single line of n space-separated integers.

Sample Input

```
2
0 2 10
5 3 5
```

Sample Output

```
2 6 14 30 62 126 254 510 1022 2046
8 14 26 50 98
```

Explanation

We have two queries:

1. We use $a = 0$, $b = 2$, and $n = 10$ to produce some series $s_0, s_1, \ldots, s_{n-1}$:
   - $s_0 = 0 + 1 \cdot 2 = 2$

Change Theme Language Java 7

```java
import java.util.*;
import java.io.*;

class Solution{
public static void main(String[] argh){
Scanner in=new Scanner(System.in);
int t=in.nextInt();
for(int i=0;i<t;i++){
int a=in.nextInt();
int b=in.nextInt();
int n=in.nextInt();
int sum=a;
int power=1;
for(int j=0;j<n;j++){
sum=sum+(power*b);
System.out.print(sum+" ");
power=power*2;
}
System.out.println();
}
in.close();
}
}
/*BHANUTEJA REDDY*/
```

Upload Code as File | Test against custom input

HackerRank | Prepare > Java > Introduction > Java Datatypes

Problem | Submissions | Leaderboard | Discussions | Editorial

Java has 8 primitive data types; char, boolean, byte, short, int, long, float, and double. For this exercise, we'll work with the primitives used to hold integer values (byte, short, int, and long):

- A byte is an 8-bit signed integer.
- A short is a 16-bit signed integer.
- An int is a 32-bit signed integer.
- A long is a 64-bit signed integer.

Given an input integer, you must determine which primitive data types are capable of properly storing that input.

To get you started, a portion of the solution is provided for you in the editor.

Reference: https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html

Input Format

The first line contains an integer, T, denoting the number of test cases.
Each test case, T, is comprised of a single line with an integer, n, which can be arbitrarily large or small.

Output Format

For each input variable n and appropriate primitive $dataType$, you must determine if the given primitives are capable of storing it. If yes, then print:

```
n can be fitted in:
* dataType
```

If there is more than one appropriate data type, print each one on its own line and order them by size (i.e.: $byte < short < int < long$).

If the number cannot be stored in one of the four aforementioned primitives, print the line:

```
n can't be fitted anywhere.
```

Sample Input

```
5
-150
150000
1500000000
213333333333333333333333333333333333
```

Change Theme Language Java 7

```java
import java.util.*;
import java.io.*;

class Solution{
public static void main(String[] argh){
Scanner sc=new Scanner(System.in);
int t=sc.nextInt();

for(int i=0;i<t;i++){
try{
long x=sc.nextLong();
System.out.println(x+" can be fitted in:");
if(x>=-128 && x<=127)
System.out.println("* byte");
if(x>=-32768 && x<=32767)
System.out.println("* short");
if(x>=-2147483648L && x<=2147483647L)
System.out.println("* int");
System.out.println("* long");
}
catch(Exception e){
System.out.println(sc.next()+" can't be fitted anywhere.");
}
}
}
}
/*BHANUTEJA REDDY*/
```

Upload Code as File | Test against custom input | Run

HackerRank | Prepare > Java > Introduction > Java End-of-file

"In computing, End Of File (commonly abbreviated EOF) is a condition in a computer operating system where no more data can be read from a data source." — (Wikipedia: End-of-file)

The challenge here is to read n lines of input until you reach EOF, then number and print all n lines of content.

Hint: Java's Scanner.hasNext() method is helpful for this problem.

Input Format

Read some unknown n lines of input from stdin(System.in) until you reach EOF; each line of input contains a non-empty String.

Output Format

For each line, print the line number, followed by a single space, and then the line content received as input.

Sample Input

```
Hello world
I am a file
Read me until end-of-file.
```

Sample Output

```
1 Hello world
2 I am a file
3 Read me until end-of-file.
```

Change Theme Language Java 7

```java
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {

        Scanner scan = new Scanner(System.in);
      int i = 0;
        while(scan.hasNext()){
       i++;
       System.out.println(i + " " + scan.nextLine());
    }
     }
}
/*BHANUTEJA REDDY*/
```

HackerRank | Prepare > Java > Introduction > Java Static Initializer Block

Static initialization blocks are executed when the class is loaded, and you can initialize static variables in those blocks.

It's time to test your knowledge of Static initialization blocks. You can read about it here.

You are given a class Solution with a main method. Complete the given code so that it outputs the area of a parallelogram with breadth B and height H. You should read the variables from the standard input.

If $B \leq 0$ or $H \leq 0$, the output should be "java.lang.Exception: Breadth and height must be positive" without quotes.

Input Format

There are two lines of input. The first line contains B: the breadth of the parallelogram. The next line contains H: the height of the parallelogram.

Constraints

- $-100 \leq B \leq 100$
- $-100 \leq H \leq 100$

Output Format

If both values are greater than zero, then the main method must output the area of the parallelogram. Otherwise, print "java.lang.Exception: Breadth and height must be positive" without quotes.

Sample input 1

```
1
3
```

Sample output 1

```
3
```

Sample input 2

```
-1
2
```

Change Theme Language Java 7

```java
import java.io.*;...
    static int B;
     static int H;
     static boolean flag = true;

     static {
         Scanner sc = new Scanner(System.in);

         B = sc.nextInt();
         H = sc.nextInt();

         if (B <= 0 || H <= 0) {
             flag = false;
             System.out.println("java.lang.Exception: Breadth and height must be positive");
         }
     }
/*BHANUTEJA REDDY*/
public static void main(String[] args){...
```

HackerRank | Prepare > Java > Introduction > Java Int to String

You are given an integer n, you have to convert it into a string.

Please complete the partially completed code in the editor. If your code successfully converts n into a string s the code will print "Good job". Otherwise it will print "Wrong answer".

n can range between -100 to 100 inclusive.

Sample Input 0

```
100
```

Sample Output 0

```
Good job
```

Change Theme Language Java 7

```java
import java.util.*;
import java.security.*;
public class Solution {
 public static void main(String[] args) {

  DoNotTerminate.forbidExit();

  try {
   Scanner in = new Scanner(System.in);
   int n = in .nextInt();
   in.close();
   //String s=???; Complete this line below
String s = Integer.toString(n);
/*BHANUTEJA REDDY*/

   if (n == Integer.parseInt(s)) {
    System.out.println("Good job");
   } else {
    System.out.println("Wrong answer.");
   }
  } catch (DoNotTerminate.ExitTrappedException e) {
   System.out.println("Unsuccessful Termination!!");
  }
 }
}

//The following class will prevent you from terminating the code using exit(0)!
class DoNotTerminate {

 public static class ExitTrappedException extends SecurityException {

  private static final long serialVersionUID = 1;
 }

 public static void forbidExit() {
  final SecurityManager securityManager = new SecurityManager() {
   @Override
   public void checkPermission(Permission permission) {
```

Line: 14 Col: 20

HackerRank | Prepare > Java > Introduction > Java Date and Time

Exit Full Screen View

Problem | Submissions | Leaderboard | Discussions | Editorial

The Calendar class is an abstract class that provides methods for converting between a specific instant in time and a set of calendar fields such as YEAR, MONTH, DAY_OF_MONTH, HOUR, and so on, and for manipulating the calendar fields, such as getting the date of the next week.

You are given a date. You just need to write the method, $getDay$, which returns the day on that date. To simplify your task, we have provided a portion of the code in the editor.

Example

$month = 8$

$day = 14$

$year = 2017$

The method should return $MONDAY$ as the day on that date.

Change Theme Language Java 7

```java
import java.io.*; ...

class Result {

    /*
     * Complete the 'findDay' function below.
     *
     * The function is expected to return a STRING.
     * The function accepts following parameters:
     *  1. INTEGER month
     *  2. INTEGER day
     *  3. INTEGER year
     */

     public static String findDay(int month, int day, int year) {

       Calendar cal = Calendar.getInstance();
       cal.set(Calendar.MONTH,month-1);
       cal.set(Calendar.DAY_OF_MONTH,day);
       cal.set(Calendar.YEAR,year);
String dayOfWeek = cal.getDisplayName(Calendar.DAY_OF_WEEK, Calendar.LONG, Locale.US).toUpperCase();
       return dayOfWeek;

    }

}
/*BHANUTEJA REDDY*/
public class Solution { ...
```

Line: 33 Col: 20

HackerRank | Prepare > Java > Introduction > Java Currency Formatter

Exit Full Sc

Problem | Submissions | Leaderboard | Discussions | Editorial

Given a double-precision number, $payment$, denoting an amount of money, use the NumberFormat class' getCurrencyInstance method to convert $payment$ into the US, Indian, Chinese, and French currency formats. Then print the formatted values as follows:

```
US: formattedPayment
India: formattedPayment
China: formattedPayment
France: formattedPayment
```

where $formattedPayment$ is $payment$ formatted according to the appropriate Locale's currency.

Note: India does not have a built-in Locale, so you must construct one where the language is `en` (i.e., English).

Input Format

A single double-precision number denoting $payment$.

Constraints

- $0 \le payment \le 10^9$

Output Format

On the first line, print `US: u` where u is $payment$ formatted for US currency.

On the second line, print `India: i` where i is $payment$ formatted for Indian currency.

On the third line, print `China: c` where c is $payment$ formatted for Chinese currency.

On the fourth line, print `France: f`, where f is $payment$ formatted for French currency.

Sample Input

```
12324.134
```

Sample Output

```
US: $12,324.13
India: Rs.12,324.13
China: ￥12,324.13
France: 12 324,13 €
```

Change Theme Language Java 7

```java
import java.util.*;
import java.text.*;

import java.io.*;
import java.util.*;
import java.text.*;

public class Solution {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);
        double payment = scanner.nextDouble();
        scanner.close();

        Locale indiaLocale = new Locale("en", "IN");

        NumberFormat us = NumberFormat.getCurrencyInstance(Locale.US);
        NumberFormat india = NumberFormat.getCurrencyInstance(indiaLocale);
        NumberFormat china = NumberFormat.getCurrencyInstance(Locale.CHINA);
        NumberFormat france = NumberFormat.getCurrencyInstance(Locale.FRANCE);

        System.out.println("US: " + us.format(payment));
        System.out.println("India: " + india.format(payment));
        System.out.println("China: " + china.format(payment));
        System.out.println("France: " + france.format(payment));
    }
}
/*BHANUTEJA REDDY*/
```

Line:

Upload Code as File | Test against custom input | Run Code

HackerRank | Prepare > Java > Strings > Java Strings Introduction

Exit Full Screen View

Problem | Submissions | Leaderboard | Discussions | Editorial

> "A string is traditionally a sequence of characters, either as a literal constant or as some kind of variable." — Wikipedia: String (computer science)

This exercise is to test your understanding of Java Strings. A sample String declaration:

```
String myString = "Hello World!"
```

The elements of a String are called characters. The number of characters in a String is called the length, and it can be retrieved with the String.length() method.

Given two strings of lowercase English letters, A and B, perform the following operations:

1. Sum the lengths of A and B.
2. Determine if A is lexicographically larger than B (i.e.: does B come before A in the dictionary?).
3. Capitalize the first letter in A and B and print them on a single line, separated by a space.

Input Format

The first line contains a string A. The second line contains another string B. The strings are comprised of only lowercase English letters.

Output Format

There are three lines of output:

For the first line, sum the lengths of A and B.

For the second line, write `Yes` if A is lexicographically greater than B otherwise print `No` instead.

For the third line, capitalize the first letter in both A and B and print them on a single line, separated by a space.

Sample Input 0

```
hello
java
```

Sample Output 0

```
9
```

Change Theme Language Java 7

```java
import java.io.*;
import java.util.*;

public class Solution {

    public static void main(String[] args) {

        Scanner sc=new Scanner(System.in);
        String A=sc.next();
        String B=sc.next();
       System.out.println(A.length()+B.length());
System.out.println(A.compareTo(B)>0?"Yes":"No");
System.out.println(A.substring(0, 1).toUpperCase()+A.substring(1, A.length())+" "+B.substring(0, 1).
toUpperCase()+B.substring(1, B.length()));

    }
}
/*BHANUTEJA REDDY*/
```

Line: 17 Col: 20

HackerRank | Prepare > Java > Strings > Java Substring

Problem | Submissions | Leaderboard | Discussions | Editorial

Given a string, s, and two indices, $start$ and end, print a substring consisting of all characters in the inclusive range from $start$ to $end - 1$. You'll find the String class' substring method helpful in completing this challenge.

Input Format

The first line contains a single string denoting s.
The second line contains two space-separated integers denoting the respective values of $start$ and end.

Constraints

- $1 \le |s| \le 100$
- $0 \le start < end \le n$
- String s consists of English alphabetic letters (i.e., $[a - zA - Z]$) only.

Output Format

Print the substring in the inclusive range from $start$ to $end - 1$.

Sample Input

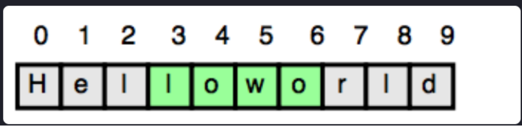
```
Helloworld
3 7
```

Sample Output

```
lowo
```

Explanation

In the diagram below, the substring is highlighted in green:



Change Theme Language Java 7

```java
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        String S = in.next();
        int start = in.nextInt();
        int end = in.nextInt();
        System.out.println(S.substring(start,end));
    }
}
/*BHANUTEJA REDDY*/
```

HackerRank | Prepare > Java > Strings > Java Substring Comparisons

Problem | Submissions | Leaderboard | Discussions | Editorial

We define the following terms:

- Lexicographical Order, also known as alphabetic or dictionary order, orders characters as follows:

$$A < B < \ldots < Y < Z < a < b < \ldots < y < z$$

  For example, `ball < cat`, `dog < dorm`, `Happy < happy`, `Zoo < ball`.
- A substring of a string is a contiguous block of characters in the string. For example, the substrings of abc are a, b, c, ab, bc, and abc.

Given a string, s, and an integer, k, complete the function so that it finds the lexicographically smallest and largest substrings of length k.

Function Description

Complete the getSmallestAndLargest function in the editor below.

getSmallestAndLargest has the following parameters:

- string s: a string
- int k: the length of the substrings to find

Returns

- string: the string ' + "\n" + ' where and are the two substrings

Input Format

The first line contains a string denoting s.
The second line contains an integer denoting k.

Constraints

- $1 \le |s| \le 1000$
- s consists of English alphabetic letters only (i.e., `[a-zA-Z]`).

Sample Input 0

```
welcometojava
3
```

Sample Output 0

Change Theme Language Java 7

```java
import java.util.Scanner;...

    public static String getSmallestAndLargest(String s, int k) {
        String smallest = "";
        String largest = "";

        java.util.List<String> a = new java.util.ArrayList<>();

        for(int i=0;i<s.length()-k+1;i++){
                a.add(s.substring(i,i+k));
        }
        java.util.Collections.sort(a);
         smallest = a.get(0);
         largest = a.get(a.size()-1);
        return smallest + "\n" + largest;
    }
/*BHANUTEJA REDDY*/
...
```

HackerRank | Prepare > Java > Strings > Java String Reverse

Problem | Submissions | Leaderboard

A palindrome is a word, phrase, number, or other sequence of characters which reads the same backward or forward.

Given a string A, print Yes if it is a palindrome, print No otherwise.

Constraints

- A will consist at most 50 lower case english letters.

Sample Input

```
madam
```

Sample Output

```
Yes
```

Change Theme Language Java 7

```java
import java.io.*;
import java.util.*;

public class Solution {

    public static void main(String[] args) {

        Scanner sc=new Scanner(System.in);
        String A=sc.next();
         String reversed = new StringBuilder(A).reverse().toString();
        if (A.equals(reversed)) {
            System.out.println("Yes");
        } else {
            System.out.println("No");
        }
    }
}
/*BHANUTEJA REDDY*/
```

HackerRank | Prepare > Java > Strings > Java Anagrams

Exit Full Screen View

Problem

Submissions

Leaderboard

Discussions

Editorial

Two strings, a and b, are called anagrams if they contain all the same characters in the same frequencies. For this challenge, the test is not case-sensitive. For example, the anagrams of CAT are CAT, ACT, tac, TCA, aTC, and CtA.

Function Description

Complete the isAnagram function in the editor.

isAnagram has the following parameters:

- string a: the first string
- string b: the second string

Returns

- boolean: If a and b are case-insensitive anagrams, return true. Otherwise, return false.

Input Format

The first line contains a string a.

The second line contains a string b.

Constraints

- $1 \leq length(a), length(b) \leq 50$
- Strings a and b consist of English alphabetic characters.
- The comparison should NOT be case sensitive.

Sample Input 0

```
anagram
margana
```

Sample Output 0

```
Anagrams
```

Explanation 0

Character	Frequency: anagram	Frequency: margana
A or a	3	3
G or g	1	1

Change Theme Language Java 7

```java
import java.util.Scanner;···
    static boolean isAnagram(String a, String b) {
 a = a.toLowerCase();
        b = b.toLowerCase();
        if (a.length() != b.length()) return false;
        int[] count = new int[26];
        for (char c : a.toCharArray()) count[c - 'a']++;
        for (char c : b.toCharArray()) count[c - 'a']--;
        for (int i : count) {
            if (i != 0) return false;
        }

        return true;
    }
    public static void main(String[] args) {···
```

Line: 4 Col:

Upload Code as File   Test against custom input   Run Code   Submit Code

HackerRank | Prepare > Java > Strings > Java String Tokens

Exit Full

Problem

Submissions

Leaderboard

Discussions

Editorial

Given a string, s, matching the regular expression [A-Za-z !,?._'@]+, split the string into tokens. We define a token to be one or more consecutive English alphabetic letters. Then, print the number of tokens, followed by each token on a new line.

Note: You may find the String.split method helpful in completing this challenge.

Input Format

A single string, s.

Constraints

- $1 \leq$ length of $s \leq 4 \cdot 10^5$
- s is composed of any of the following: English alphabetic letters, blank spaces, exclamation points (!), commas (,), question marks (?), periods (.), underscores (_), apostrophes ('), and at symbols (@).

Output Format

On the first line, print an integer, n, denoting the number of tokens in string s (they do not need to be unique). Next, print each of the n tokens on a new line in the same order as they appear in input string s.

Sample Input

```
He is a very very good boy, isn't he?
```

Sample Output

```
10
He
is
a
very
very
good
boy
isn
t
he
```

Explanation

Change Theme Language Java 7

```java
import java.io.*;
import java.util.*;

public class Solution {

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        String s = scan.nextLine().trim();

        if (s == null || s.isEmpty()) {
            System.out.println(0);
        } else {
            String[] tokens = s.split("[^a-zA-Z]+");
            if (tokens.length > 0 && tokens[0].isEmpty()) {
                tokens = Arrays.copyOfRange(tokens, 1, tokens.length);
            }
            System.out.println(tokens.length);
            for (String token : tokens) {
                System.out.println(token);
            }
        }

        scan.close();
    }
}
/*BHANUTEJA REDDY*/
```

Line:

Upload Code as File   Test against custom input   Run Code

HackerRank | Prepare > Java > Strings > Pattern Syntax Checker

Problem | Submissions | Leaderboard | Discussions

Using **Regex**, we can easily match or search for patterns in a text. Before searching for a pattern, we have to specify one using some well-defined syntax.

In this problem, you are given a pattern. You have to check whether the syntax of the given pattern is valid.

Note: In this problem, a regex is only valid if you can compile it using the Pattern.compile method.

Input Format

The first line of input contains an integer N, denoting the number of test cases. The next N lines contain a string of any printable characters representing the pattern of a regex.

Output Format

For each test case, print `Valid` if the syntax of the given pattern is correct. Otherwise, print `Invalid`. Do not print the quotes.

Sample Input

```
3
([A-Z])(.+)
[AZ[a-z](a-z)
batcatpat(nat
```

Sample Output

```
Valid
Invalid
Invalid
```

Change Theme Language

```java
import java.util.Scanner;
import java.util.regex.*;

public class Solution
{
    public static void main(String[] args){
        Scanner in = new Scanner(System.in);
        int testCases = Integer.parseInt(in.nextLine());
        while(testCases>0){
            String pattern = in.nextLine();
             try {
        Pattern.compile(pattern);
        System.out.println("Valid");
} catch (Exception e) {
        System.out.println("Invalid");
}
testCases--;
        }
    }
}
/*BHANUTEJA REDDY*/
```

HackerRank | Prepare > Java > Strings > Java Regex

Exit Full Screen View

Problem | Submissions | Leaderboard | Discussions | Editorial

Write a class called MyRegex which will contain a string pattern. You need to write a regular expression and assign it to the pattern such that it can be used to validate an IP address. Use the following definition of an IP address:

```
IP address is a string in the form "A.B.C.D", where the value of A, B, C, and D may range
```

Some valid IP address:

```
000.12.12.034
121.234.12.12
23.45.12.56
```

Some invalid IP address:

```
000.12.234.23.23
666.666.23.23
.213.123.23.32
23.45.22.32.
I.Am.not.an.ip
```

In this problem you will be provided strings containing any combination of ASCII characters. You have to write a regular expression to find the valid IPs.

Just write the MyRegex class which contains a String *pattern*. The string should contain the correct regular expression.

(MyRegex class MUST NOT be public)

Sample Input

```
000.12.12.034
121.234.12.12
23.45.12.56
00.12.123.123123.123
122.23
Hello.IP
```

Change Theme Language Java 7

```java
import java.util.regex.Matcher;
import java.util.regex.Pattern;
import java.util.Scanner;

class Solution{

    public static void main(String[] args){
        Scanner in = new Scanner(System.in);
        while(in.hasNext()){
            String IP = in.next();
            System.out.println(IP.matches(new MyRegex().pattern));
        }

    }
}
class MyRegex {
    String pattern = "(([01]?\\d\\d?|2[0-4]\\d|25[0-5])\\.){3}([01]?\\d\\d?|2[0-4]\\d|25[0-5])";
}
/*BHANUTEJA REDDY*/
```

Line: 19 Col: 20

HackerRank | Prepare > Java > Object Oriented Programming > Java Interface

Exit Full Screen View

Problem | Submissions | Leaderboard | Discussions | Editorial

A Java interface can only contain method signatures and fields. The interface can be used to achieve polymorphism. In this problem, you will practice your knowledge on interfaces.

You are given an interface AdvancedArithmetic which contains a method signature int divisor_sum(int n). You need to write a class called MyCalculator which implements the interface.

divisorSum function just takes an integer as input and return the sum of all its divisors. For example divisors of 6 are 1, 2, 3 and 6, so divisor_sum should return 12. The value of n will be at most 1000.

Read the partially completed code in the editor and complete it. You just need to write the MyCalculator class only. Your class shouldn't be public.

Sample Input

```
6
```

Sample Output

```
I implemented: AdvancedArithmetic
12
```

Explanation

Divisors of 6 are 1,2,3 and 6. 1+2+3+6=12.

Change Theme Language Java 7

```java
import java.util.*;...
class MyCalculator implements AdvancedArithmetic {
    public int divisor_sum(int n) {
        if (n <= 1) { return n; }

        int res = n + 1;
        for (int i = 2; i < n; i++) {
            if (n % i == 0) {
                res += i;
            }
        }

        return res;
    }
}
/*BHANUTEJA REDDY*/
class Solution{
    public static void main(String []args){
        MyCalculator my_calculator = new MyCalculator();
        System.out.print("I implemented: ");
        ImplementedInterfaceNames(my_calculator);
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        System.out.print(my_calculator.divisor_sum(n) + "\n");
        sc.close();
    }
    /*
     * ImplementedInterfaceNames method takes an object and prints the name of the interfaces it implemented
     */
    static void ImplementedInterfaceNames(Object o){
        Class[] theInterfaces = o.getClass().getInterfaces();
        for (int i = 0; i < theInterfaces.length; i++){
            String interfaceName = theInterfaces[i].getName();
            System.out.println(interfaceName);
        }
    }
}
```

HackerRank | Prepare > Java > Object Oriented Programming > Java Method Overriding

Exit Full Sc

Problem

Submissions

Leaderboard

Discussions

Editorial

When a subclass inherits from a superclass, it also inherits its methods; however, it can also override the superclass methods (as well as declare and implement new ones). Consider the following Sports class:

```
class Sports{
    String getName(){
        return "Generic Sports";
    }
    void getNumberOfTeamMembers(){
        System.out.println( "Each team has n players in " + getName() );
    }
}
```

Next, we create a Soccer class that inherits from the Sports class. We can override the getName method and return a different, subclass-specific string:

```
class Soccer extends Sports{
    @Override
    String getName(){
        return "Soccer Class";
    }
}
```

Note: When overriding a method, you should precede it with the `@Override` annotation. The parameter(s) and return type of an overridden method must be exactly the same as those of the method inherited from the supertype.

Task

Complete the code in your editor by writing an overridden getNumberOfTeamMembers method that prints the same statement as the superclass' getNumberOfTeamMembers method, except that it replaces n with 11 (the number of players on a Soccer team).

Output Format

When executed, your completed code should print the following:

```
Generic Sports
Each team has n players in Generic Sports
Soccer Class
Each team has 11 players in Soccer Class
```

Change Theme Language Java 7

```java
import java.util.*;
class Sports{

    String getName(){
        return "Generic Sports";
    }

    void getNumberOfTeamMembers(){
        System.out.println( "Each team has n players in " + getName() );
    }
}

class Soccer extends Sports{
    @Override
    String getName(){
        return "Soccer Class";
    }
   @Override
    void getNumberOfTeamMembers() {
        System.out.println("Each team has 11 players in " + getName());
    }
/*BHANUTEJA REDDY*/
}...
```

Line:

Upload Code as File | Test against custom input | Run Code

HackerRank | Prepare > Java > Object Oriented Programming > Java Method Overriding 2 (Super Keyword)

Exit Full Scr

Problem

Submissions

Leaderboard

Discussions

When a method in a subclass overrides a method in superclass, it is still possible to call the overridden method using **super** keyword. If you write super.func() to call the function func(), it will call the method that was defined in the superclass.

You are given a partially completed code in the editor. Modify the code so that the code prints the following text:

```
Hello I am a motorcycle, I am a cycle with an engine.
My ancestor is a cycle who is a vehicle with pedals.
```

Change Theme Language Java 7

```java
import java.util.*;
import java.io.*;

class BiCycle{
    String define_me(){
        return "a vehicle with pedals.";
    }
}

class MotorCycle extends BiCycle{
    String define_me(){
        return "a cycle with an engine.";
    }

    MotorCycle(){
        System.out.println("Hello I am a motorcycle, I am "+ define_me());
/*BHANUTEJA REDDY*/
        String temp=super.define_me();

        System.out.println("My ancestor is a cycle who is "+ temp );...
```